

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

data.csv

data.csv(text/csv) - 528805 bytes, last modified: 31/07/2026 - 100% done
Saving data.csv to data.csv

```
df = pd.read_csv("data.csv", sep=";")
```

```
# Remove spaces from column names if any
df.columns = df.columns.str.strip()
```

```
print(df.head())
```

```
Marital status  Application mode  Application order  Course \
0              1              17              5      171
1              1              15              1     9254
2              1               1              5     9070
3              1              17              2     9773
4              2              39              1     8014

Daytime/evening attendance  Previous qualification \
0              1              1
1              1              1
2              1              1
3              1              1
4              0              1

Previous qualification (grade)  Nacionalidad  Mother's qualification \
0              122.0              1              19
1              160.0              1              1
2              122.0              1              37
3              122.0              1              38
4              100.0              1              37

Father's qualification  ...  Curricular units 2nd sem (credited) \
0              12  ...              0
1              3  ...              0
2              37  ...              0
3              37  ...              0
4              38  ...              0

Curricular units 2nd sem (enrolled) \
0              0
1              6
2              6
3              6
4              6

Curricular units 2nd sem (evaluations) \
0              0
1              6
2              0
3             10
4              6

Curricular units 2nd sem (approved)  Curricular units 2nd sem (grade) \
0              0              0.000000
1              6             13.666667
2              0              0.000000
3              5             12.400000
4              6             13.000000

Curricular units 2nd sem (without evaluations)  Unemployment rate \
0              0              10.8
```

1	0	13.9
2	0	10.8
3	0	9.4
4	0	13.9

Inflation rate	GDP	Target
----------------	-----	--------

```
print(df.columns.tolist())
```

```
['Marital status', 'Application mode', 'Application order', 'Course', 'Daytime/evening attendance', 'Previous qualification']
```

```
print("Shape:", df.shape)
```

```
df.info()
```

```
Shape: (4424, 37)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4424 entries, 0 to 4423
Data columns (total 37 columns):
#   Column                                                                 Non-Null Count  Dtype
---  -
0   Marital status                                                         4424 non-null   int64
1   Application mode                                                         4424 non-null   int64
2   Application order                                                         4424 non-null   int64
3   Course                                                                  4424 non-null   int64
4   Daytime/evening attendance                                              4424 non-null   int64
5   Previous qualification                                                  4424 non-null   int64
6   Previous qualification (grade)                                          4424 non-null   float64
7   Nacionality                                                            4424 non-null   int64
8   Mother's qualification                                                  4424 non-null   int64
9   Father's qualification                                                  4424 non-null   int64
10  Mother's occupation                                                     4424 non-null   int64
11  Father's occupation                                                     4424 non-null   int64
12  Admission grade                                                         4424 non-null   float64
13  Displaced                                                               4424 non-null   int64
14  Educational special needs                                              4424 non-null   int64
15  Debtor                                                                  4424 non-null   int64
16  Tuition fees up to date                                                4424 non-null   int64
17  Gender                                                                  4424 non-null   int64
18  Scholarship holder                                                      4424 non-null   int64
19  Age at enrollment                                                       4424 non-null   int64
20  International                                                            4424 non-null   int64
21  Curricular units 1st sem (credited)                                    4424 non-null   int64
22  Curricular units 1st sem (enrolled)                                    4424 non-null   int64
23  Curricular units 1st sem (evaluations)                                4424 non-null   int64
24  Curricular units 1st sem (approved)                                    4424 non-null   int64
25  Curricular units 1st sem (grade)                                       4424 non-null   float64
26  Curricular units 1st sem (without evaluations)                        4424 non-null   int64
27  Curricular units 2nd sem (credited)                                    4424 non-null   int64
28  Curricular units 2nd sem (enrolled)                                    4424 non-null   int64
29  Curricular units 2nd sem (evaluations)                                4424 non-null   int64
30  Curricular units 2nd sem (approved)                                    4424 non-null   int64
31  Curricular units 2nd sem (grade)                                       4424 non-null   float64
32  Curricular units 2nd sem (without evaluations)                        4424 non-null   int64
33  Unemployment rate                                                       4424 non-null   float64
34  Inflation rate                                                         4424 non-null   float64
35  GDP                                                                    4424 non-null   float64
36  Target                                                                  4424 non-null   object
dtypes: float64(7), int64(29), object(1)
memory usage: 1.2+ MB
```

```
print(df.isnull().sum())
```

Marital status	0
Application mode	0
Application order	0
Course	0
Daytime/evening attendance	0
Previous qualification	0
Previous qualification (grade)	0
Nacionality	0
Mother's qualification	0
Father's qualification	0
Mother's occupation	0
Father's occupation	0
Admission grade	0
Displaced	0
Educational special needs	0
Debtor	0
Tuition fees up to date	0
Gender	0
Scholarship holder	0
Age at enrollment	0
International	0
Curricular units 1st sem (credited)	0
Curricular units 1st sem (enrolled)	0
Curricular units 1st sem (evaluations)	0

```

Curricular units 1st sem (approved)      0
Curricular units 1st sem (grade)         0
Curricular units 1st sem (without evaluations) 0
Curricular units 2nd sem (credited)      0
Curricular units 2nd sem (enrolled)      0
Curricular units 2nd sem (evaluations)   0
Curricular units 2nd sem (approved)      0
Curricular units 2nd sem (grade)         0
Curricular units 2nd sem (without evaluations) 0
Unemployment rate                        0
Inflation rate                          0
GDP                                      0
Target                                  0
dtype: int64

```

```
print(df["Target"].value_counts())
```

```

Target
Graduate    2209
Dropout     1421
Enrolled     794
Name: count, dtype: int64

```

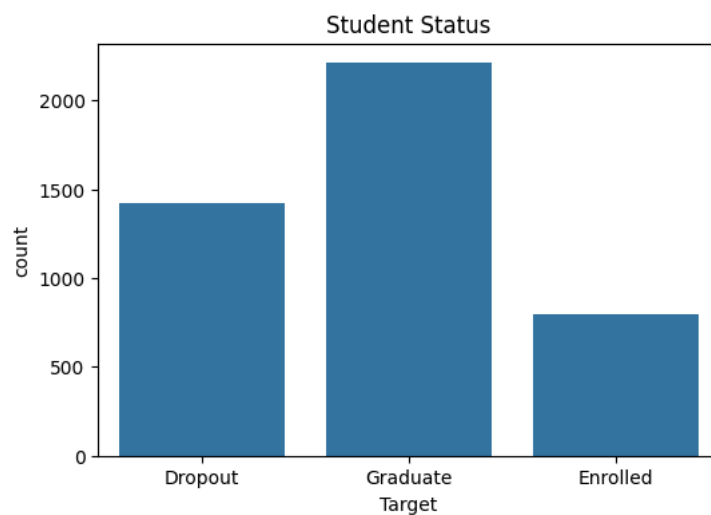
```

plt.figure(figsize=(6,4))

sns.countplot(data=df, x="Target")

plt.title("Student Status")
plt.show()

```



```

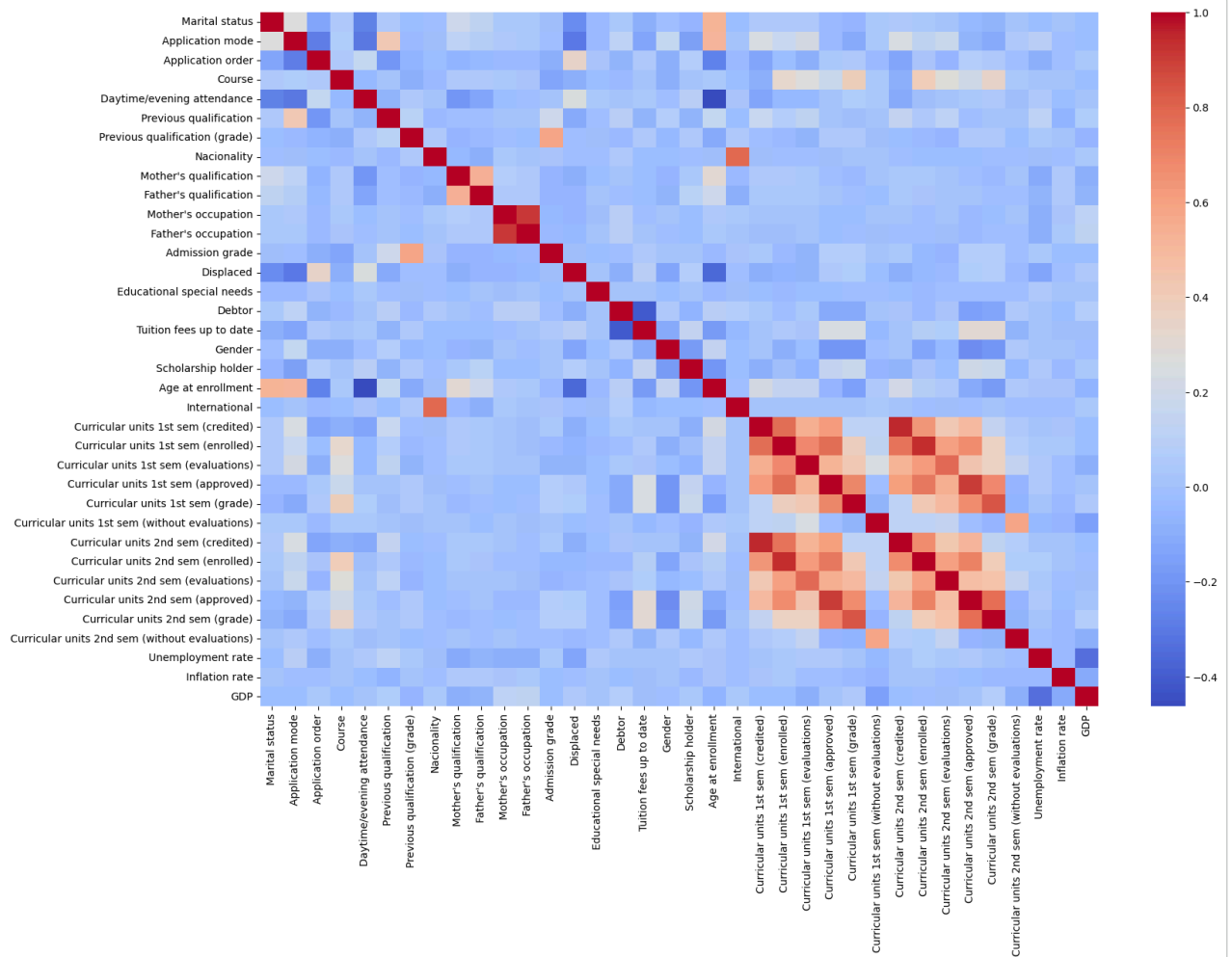
plt.figure(figsize=(18,12))

corr = df.drop(columns=["Target"]).corr()

sns.heatmap(corr, cmap="coolwarm")

plt.show()

```



```
encoder = LabelEncoder()

df["Target"] = encoder.fit_transform(df["Target"])

print(encoder.classes_)

['Dropout' 'Enrolled' 'Graduate']
```

```
X = df.drop("Target", axis=1)

y = df["Target"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
rf = RandomForestClassifier(random_state=42)

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))

Random Forest Accuracy: 0.7672316384180791
```

```
dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train, y_train)

dt_pred = dt.predict(X_test)

print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
```

Decision Tree Accuracy: 0.6971751412429379

```
lr = LogisticRegression(max_iter=5000)

lr.fit(X_train, y_train)

lr_pred = lr.predict(X_test)

print("Logistic Regression Accuracy:", accuracy_score(y_test, lr_pred))
```

Logistic Regression Accuracy: 0.7638418079096045
 /usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
 STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
 Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
 n_iter_i = _check_optimize_result(

```
knn = KNeighborsClassifier()

knn.fit(X_train, y_train)

knn_pred = knn.predict(X_test)

print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
```

KNN Accuracy: 0.5774011299435028

```
results = pd.DataFrame({

    "Model":[
        "Random Forest",
        "Decision Tree",
        "Logistic Regression",
        "KNN"
    ],

    "Accuracy":[

        accuracy_score(y_test, rf_pred),

        accuracy_score(y_test, dt_pred),

        accuracy_score(y_test, lr_pred),

        accuracy_score(y_test, knn_pred)

    ]

})

results.sort_values(by="Accuracy", ascending=False)
```

	Model	Accuracy	
0	Random Forest	0.767232	
2	Logistic Regression	0.763842	
1	Decision Tree	0.697175	
3	KNN	0.577401	

```
print(classification_report(y_test, rf_pred, target_names=encoder.classes_))
```

	precision	recall	f1-score	support
Dropout	0.81	0.75	0.78	284
Enrolled	0.57	0.37	0.45	159
Graduate	0.78	0.92	0.85	442
accuracy			0.77	885

macro avg	0.72	0.68	0.69	885
weighted avg	0.75	0.77	0.75	885

```
cm = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(6,5))

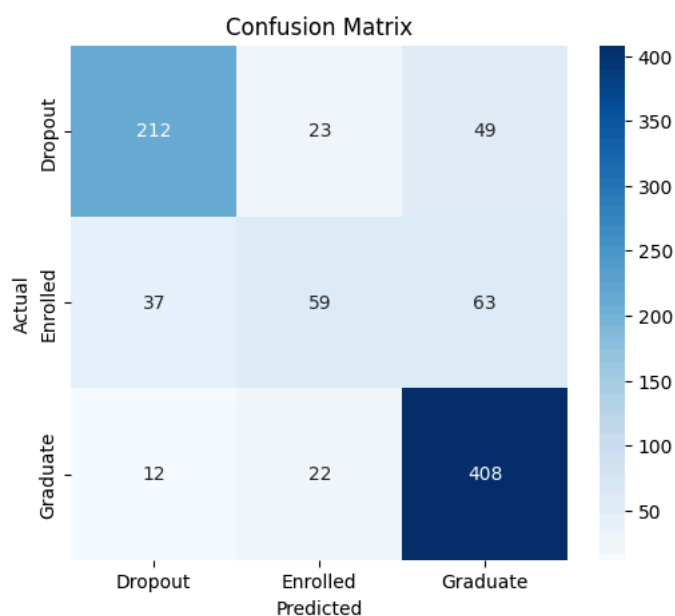
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=encoder.classes_,
    yticklabels=encoder.classes_
)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()
```



```
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf.feature_importances_
})

importance = importance.sort_values(by="Importance", ascending=False)

importance.head(10)
```

	Feature	Importance	
30	Curricular units 2nd sem (approved)	0.142283	
31	Curricular units 2nd sem (grade)	0.109008	
24	Curricular units 1st sem (approved)	0.091937	
25	Curricular units 1st sem (grade)	0.059588	
12	Admission grade	0.043575	
19	Age at enrollment	0.040287	
16	Tuition fees up to date	0.038903	
29	Curricular units 2nd sem (evaluations)	0.038200	
23	Curricular units 1st sem (evaluations)	0.037504	
6	Previous qualification (grade)	0.037498	

Next steps:

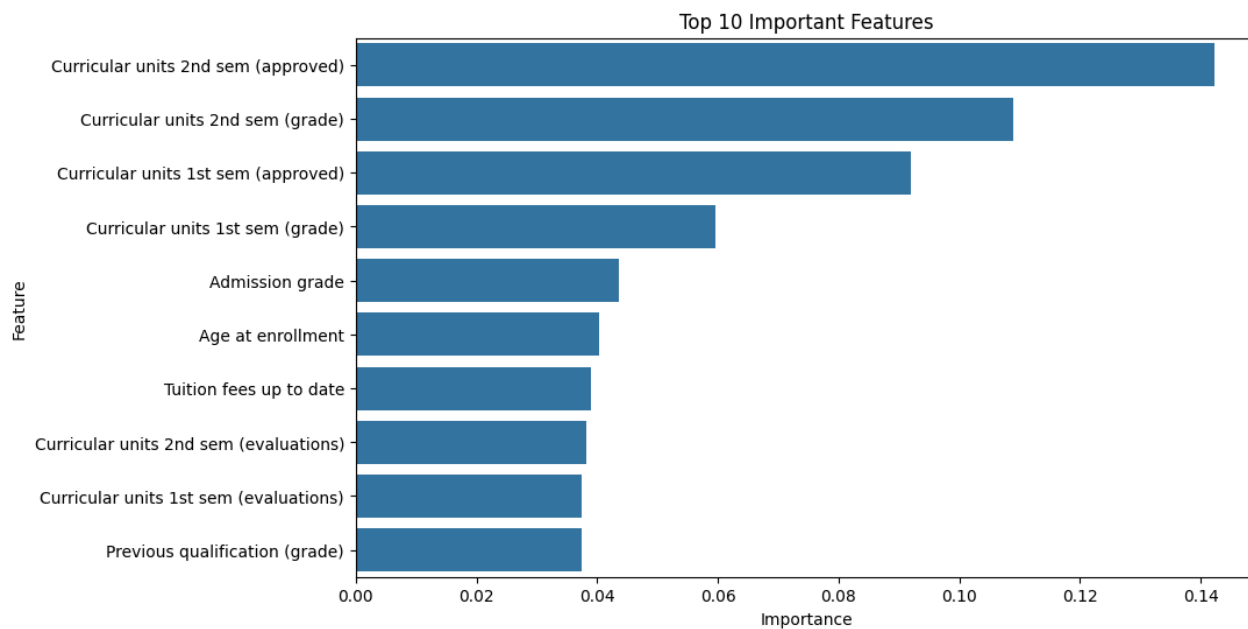
[Generate code with importance](#)[New interactive sheet](#)

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=importance.head(10),
    x="Importance",
    y="Feature"
)

plt.title("Top 10 Important Features")

plt.show()
```



```
sample = X.iloc[[0]]

prediction = rf.predict(sample)
```